ACTIVITY-BASED LEARNING ASSIGNMENT

PYTHON PROGRAMMING

Branch / Section: CSM 1

Name: G. Nandita Priya

Subject: Python Programming

Programs • Algorithms • Pseudocode • Standard-symbol flowcharts • Test cases • Sample output

Activity 1 – University Scholarship Allocation (Case Study 1)

Problem statement: Check scholarship eligibility using marks, attendance, and annual family income.

Algorithm

1. Start.
2. Input student name, marks percentage, attendance percentage, and annual family income.
3. Check whether marks ≥ 80, attendance ≥ 75, and income ≤ ■200,000.
4. If all conditions are true, display eligible; otherwise, display not eligible.
5. Stop.

Pseudocode

```
START
INPUT name, marks, attendance, income
IF marks >= 80 AND attendance >= 75 AND income <= 200000
    PRINT "Eligible for scholarship"
ELSE
    PRINT "Not eligible for scholarship"
END IF
STOP
```

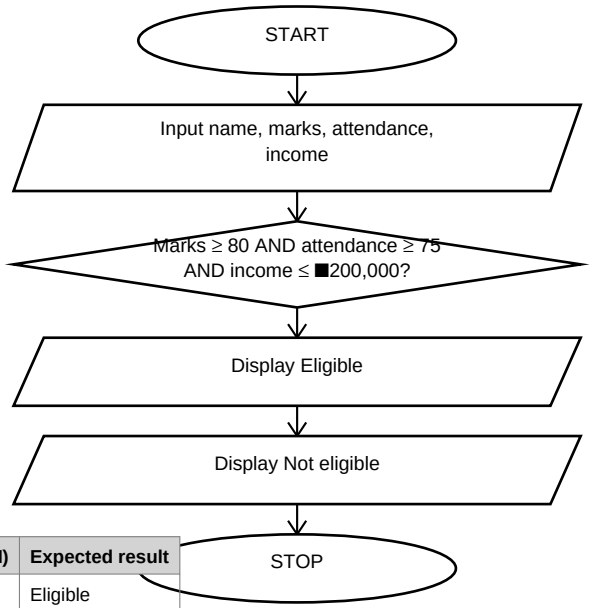
Python Program

```
name = input("Enter student name: ")
marks = float(input("Enter marks percentage: "))
attendance = float(input("Enter attendance percentage: "))
income = float(input("Enter annual family income: "))

if marks >= 80 and attendance >= 75 and income <= 200000:
    print(name, "is eligible for scholarship")
else:
    print(name, "is not eligible for scholarship")
```

Flowchart (standard symbols)

Oval = Start/Stop; parallelogram = input/output; rectangle = processing; diamond = decision.



Test Cases

Name	Marks	Attendance	Income (■)	Expected result
Ravi	90	85	150000	Eligible
Asha	79	90	150000	Not eligible
Kiran	85	74	150000	Not eligible
Meena	85	80	250000	Not eligible

Sample Output (Text Format)

```
Enter student name: Ravi
Enter marks percentage: 90
Enter attendance percentage: 85
Enter annual family income: 150000
Ravi is eligible for scholarship
```

Conclusion

Check scholarship eligibility using marks, attendance, and annual family income. The program demonstrates the Python concepts required for this activity.

Activity 1 – Assumptions

Program

Activity 1 – University Scholarship Allocation (Case Study 1)

Assumptions

Assume marks and attendance are percentages from 0 to 100. Scholarship eligibility requires marks of at least 80%, attendance of at least 75%, and annual family income not exceeding Rs. 2,00,000.

Activity 2 – Progressive Electricity Billing (Case Study 2)

Problem statement: Calculate a household electricity bill using progressive unit slabs and a fixed charge.

Algorithm

1. Start.
2. Input units consumed.
3. If units are negative, display invalid units.
4. Otherwise calculate charges: first 100 units at ■2/unit, next 100 at ■3/unit, and units above 200 at ■5/unit.
5. Add fixed charge of ■50 and display the bill.
6. Stop.

Pseudocode

```
START
INPUT units
IF units < 0
    PRINT "Invalid units"
ELSE IF units <= 100
    bill = units * 2 + 50
ELSE IF units <= 200
    bill = 100 * 2 + (units - 100) * 3 + 50
ELSE
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5 + 50
END IF
IF units >= 0
    PRINT bill
END IF
STOP
```

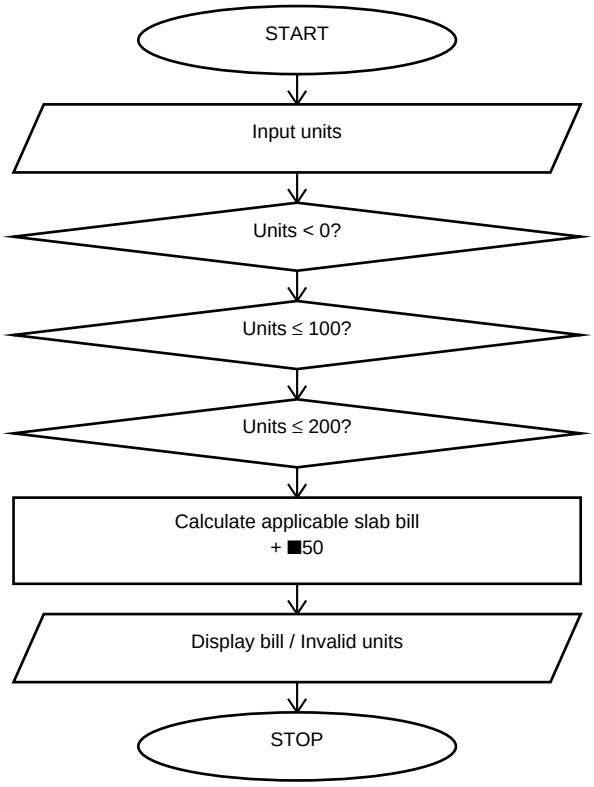
Python Program

```
units = int(input("Enter units consumed: "))

if units < 0:
    print("Invalid units")
elif units <= 100:
    bill = units * 2 + 50
    print("Total electricity bill: Rs.", bill)
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3 + 50
    print("Total electricity bill: Rs.", bill)
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5 + 50
    print("Total electricity bill: Rs.", bill)
```

Flowchart (standard symbols)

Oval = Start/Stop; parallelogram = input/output; rectangle = processing; diamond = decision.



Test Cases

Units	Expected output
-1	Invalid units
0	■50
100	■250
150	■400
250	■850

Sample Output (Text Format)

```
Enter units consumed: 150
Total electricity bill: Rs. 400
```

Conclusion

Calculate a household electricity bill using progressive unit slabs and a fixed charge. The program demonstrates the Python concepts required for this activity.

Activity 2 – Assumptions

Program

Activity 2 – Progressive Electricity Billing (Case Study 2)

Assumptions

Assume units consumed are whole numbers and cannot be negative. Charge the first 100 units at Rs. 2 per unit, the next 100 at Rs. 3 per unit, and units above 200 at Rs. 5 per unit. Add a fixed charge of Rs. 50.

Activity 3 – Employee Payroll and Deduction Analysis (Case Study 4)

Problem statement: Calculate an employee·s gross salary, deduction, and net salary from basic pay, overtime, and incentive.

Algorithm

1. Start.
2. Input basic salary, overtime amount, and incentive.
3. Calculate HRA = 20% of basic salary and DA = 10% of basic salary.
4. Calculate gross salary = basic + HRA + DA + overtime + incentive.
5. Calculate deduction = 5% of gross salary and net salary = gross – deduction.
6. Display the salary statement and stop.

Pseudocode

```
START
INPUT basic, overtime, incentive
HRA = basic * 0.20
DA = basic * 0.10
gross = basic + HRA + DA + overtime + incentive
deduction = gross * 0.05
net = gross - deduction
PRINT HRA, DA, gross, deduction, net
STOP
```

Python Program

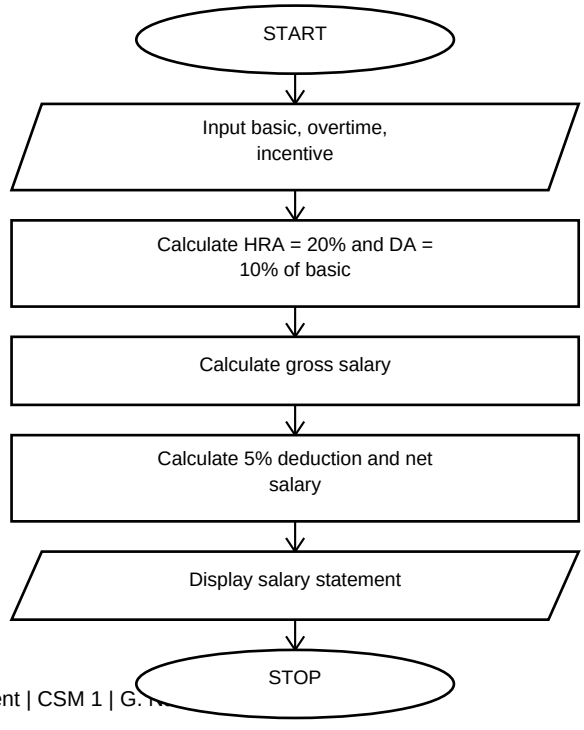
```
basic = float(input("Enter basic salary: "))
overtime = float(input("Enter overtime amount: "))
incentive = float(input("Enter incentive amount: "))

hra = basic * 0.20
da = basic * 0.10
gross = basic + hra + da + overtime + incentive
deduction = gross * 0.05
net = gross - deduction

print("HRA:", hra)
print("DA:", da)
print("Gross salary:", gross)
print("Deduction:", deduction)
print("Net salary:", net)
```

Flowchart (standard symbols)

Oval = Start/Stop; parallelogram = input/output; rectangle = processing; diamond = decision.



Test Cases

Basic	Overtime	Incentive	Expected gross	Expected net
20000	1000	500	27500	26125
10000	0	0	13000	12350
30000	2000	1000	42000	39900

Sample Output (Text Format)

```
Enter basic salary: 20000
Enter overtime amount: 1000
Enter incentive amount: 500
HRA: 4000.0
DA: 2000.0
Gross salary: 27500.0
Deduction: 1375.0
Net salary: 26125.0
```

Conclusion

Calculate an employee’s gross salary, deduction, and net salary from basic pay, overtime, and incentive. The program demonstrates the Python concepts required for this activity.

Activity 3 – Assumptions

Program

Activity 3 – Employee Payroll and Deduction Analysis (Case Study 4)

Assumptions

Assume all salary inputs are non-negative amounts. HRA is 20% and DA is 10% of basic salary. The deduction is fixed at 5% of gross salary.

Activity 4 – Retail Billing with Multiple Discount Rules (Case Study 7)

Problem statement: Prepare a two-product bill, apply a discount when the subtotal reaches a threshold, and add tax.

Algorithm

1. Start.

2. Input each product's price and quantity.

3. Calculate subtotal = price1 × quantity1 + price2 × quantity2.

4. If subtotal ≥ ■1,000, apply 10% discount; otherwise discount is zero.

5. Calculate 5% tax on the discounted amount and calculate final bill.

6. Display subtotal, discount, tax, and final bill. Stop.

Pseudocode

```
START
INPUT price1, quantity1, price2, quantity2
subtotal = price1 * quantity1 + price2 * quantity2
IF subtotal >= 1000
    discount = subtotal * 0.10
ELSE
    discount = 0
END IF
amount = subtotal - discount
tax = amount * 0.05
total = amount + tax
PRINT subtotal, discount, tax, total
STOP
```

Python Program

```
p1 = float(input("Enter first product price: "))
q1 = int(input("Enter first product quantity: "))
p2 = float(input("Enter second product price: "))
q2 = int(input("Enter second product quantity: "))

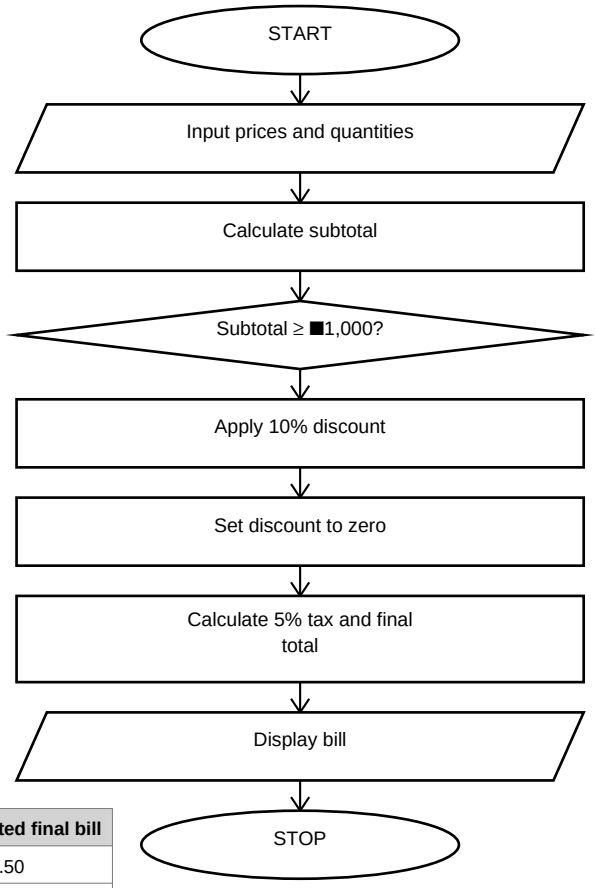
subtotal = p1 * q1 + p2 * q2
if subtotal >= 1000:
    discount = subtotal * 0.10
else:
    discount = 0

amount = subtotal - discount
tax = amount * 0.05
total = amount + tax

print("Subtotal:", subtotal)
print("Discount:", discount)
print("Tax:", tax)
print("Final bill:", total)
```

Flowchart (standard symbols)

Oval = Start/Stop; parallelogram = input/output; rectangle = processing; diamond = decision.



Test Cases

Prices and quantities	Subtotal	Expected final bill
600×1, 500×1	■1100	■1039.50
300×1, 200×1	■500	■525.00
1000×1, 0×1	■1000	■945.00

Sample Output (Text Format)

```
Enter first product price: 600
Enter first product quantity: 1
Enter second product price: 500
Enter second product quantity: 1
Subtotal: 1100.0
Discount: 110.0
Tax: 49.5
Final bill: 1039.5
```

Conclusion

Prepare a two-product bill, apply a discount when the subtotal reaches a threshold, and add tax. The program demonstrates the Python concepts required for this activity.

Activity 4 – Assumptions

Program

Activity 4 – Retail Billing with Multiple Discount Rules (Case Study 7)

Assumptions

Assume product prices are non-negative and quantities are whole numbers. Apply a 10% discount when the subtotal is Rs. 1,000 or more, then add 5% tax to the discounted amount.

Activity 5 – Examination Result and Grace-Mark Validation (Case Study 11)

Problem statement: Check a student·s result in three subjects and apply the permitted grace-mark rule.

Algorithm

1. Start.

2. Input marks for three subjects, validating each mark is between 0 and 100.

3. If a subject has 30–34 marks and grace has not been used, add 5 marks to that subject only.

4. Check whether all final marks are at least 35.

5. Display final marks and PASS or FAIL. Stop.

Pseudocode

```
START
INPUT marks for 3 subjects
IF any mark < 0 OR any mark > 100
    PRINT "Invalid marks"
ELSE
    grace_used = FALSE
    FOR each subject
        IF 30 <= mark < 35 AND grace_used = FALSE
            mark = mark + 5
            grace_used = TRUE
        END IF
    END FOR
    IF all marks >= 35
        PRINT "PASS"
    ELSE
        PRINT "FAIL"
    END IF
END IF
STOP
```

Python Program

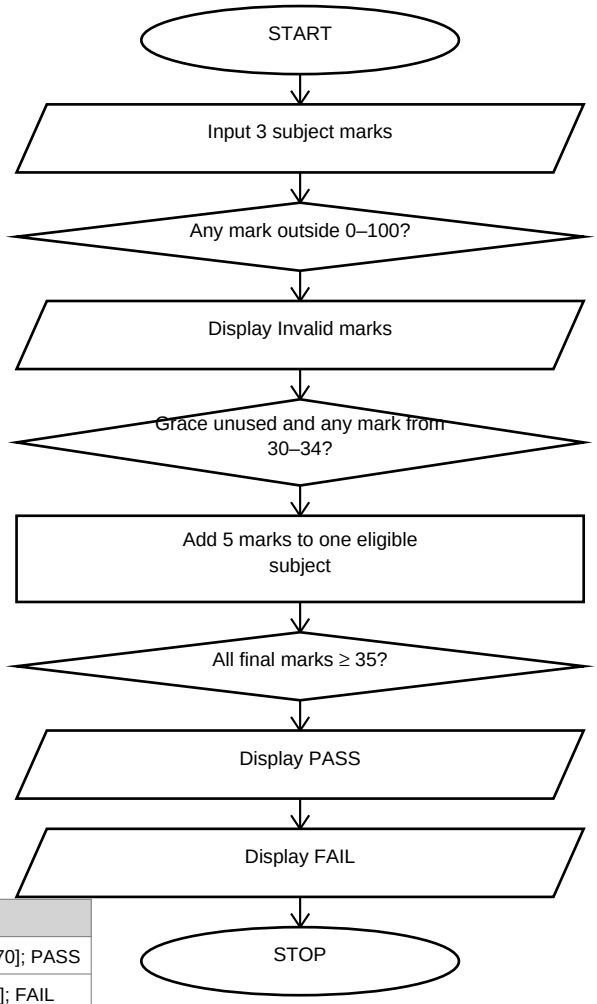
```
marks = []
for i in range(3):
    mark = int(input("Enter marks for subject " + str(i + 1) + ": "))
    if mark < 0 or mark > 100:
        print("Invalid marks")
        break
    marks.append(mark)

if len(marks) == 3:
    grace_used = False
    for i in range(3):
        if 30 <= marks[i] < 35 and not grace_used:
            marks[i] += 5
            grace_used = True
            print("Grace marks applied to subject", i + 1)

    print("Final marks:", marks)
    if all(mark >= 35 for mark in marks):
        print("Result: PASS")
    else:
        print("Result: FAIL")
```

Flowchart (standard symbols)

Oval = Start/Stop; parallelogram = input/output; rectangle = processing; diamond = decision.



Test Cases

Input marks	Expected result
32, 60, 70	Grace to subject 1; [37, 60, 70]; PASS
30, 31, 80	Grace once only; [35, 31, 80]; FAIL
-1, 60, 70	Invalid marks
35, 50, 70	No grace; PASS

Sample Output (Text Format)

```
Enter marks for subject 1: 32
Enter marks for subject 2: 60
Enter marks for subject 3: 70
Grace marks applied to subject 1
Final marks: [37, 60, 70]
Result: PASS
```

Conclusion

Check a student's result in three subjects and apply the permitted grace-mark rule. The program demonstrates the Python concepts required for this activity.

Activity 5 – Assumptions

Program

Activity 5 – Examination Result and Grace-Mark Validation (Case Study 11)

Assumptions

Assume there are exactly three subjects and each mark must be between 0 and 100. A single grace allowance of 5 marks may be added to only one subject scoring from 30 to 34. Passing requires every final subject mark to be at least 35.

Activity 6 – ATM Transaction Sequence Simulation (Case Study 13)

Problem statement: Simulate a deposit or withdrawal and update the account balance.

Algorithm

1. Start with opening balance ■5,000.
2. Input transaction type and amount.
3. If amount ≤ 0, display invalid amount.
4. For deposit, add amount to balance.
5. For withdrawal, check the ■2,000 transaction limit and available balance before subtracting.
6. Display the available balance and stop.

Pseudocode

```
START
balance = 5000
INPUT transaction, amount
IF amount <= 0
    PRINT "Invalid amount"
ELSE IF transaction = "deposit"
    balance = balance + amount
ELSE IF transaction = "withdraw"
    IF amount > 2000
        PRINT "Withdrawal limit exceeded"
    ELSE IF amount > balance
        PRINT "Insufficient balance"
    ELSE
        balance = balance - amount
        PRINT "Withdrawal successful"
    END IF
ELSE
    PRINT "Invalid transaction"
END IF
PRINT balance
STOP
```

Python Program

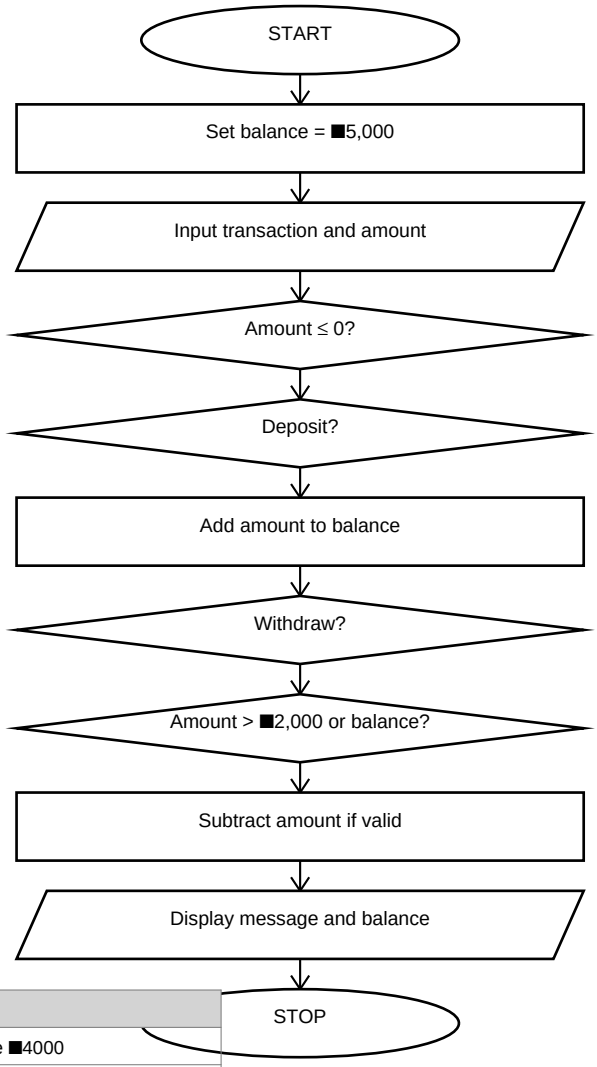
```
balance = 5000
transaction = input("Enter deposit or withdraw: ").lower()
amount = float(input("Enter amount: "))

if amount <= 0:
    print("Invalid amount")
elif transaction == "deposit":
    balance += amount
    print("Deposit successful")
elif transaction == "withdraw":
    if amount > 2000:
        print("Withdrawal limit exceeded")
    elif amount > balance:
        print("Insufficient balance")
    else:
        balance -= amount
        print("Withdrawal successful")
else:
    print("Invalid transaction")

print("Available balance:", balance)
```

Flowchart (standard symbols)

Oval = Start/Stop; parallelogram = input/output; rectangle = processing; diamond = decision.



Test Cases

Transaction	Amount	Expected result
withdraw	1000	Success; balance ■4000
deposit	500	Success; balance ■5500
withdraw	2500	Withdrawal limit exceeded; balance ■5000
withdraw	0	Invalid amount; balance ■5000

Sample Output (Text Format)

```
Enter deposit or withdraw: withdraw
Enter amount: 1000
Withdrawal successful
Available balance: 4000.0
```

Conclusion

Simulate a deposit or withdrawal and update the account balance. The program demonstrates the Python concepts required for this activity.

Activity 6 – Assumptions

Program

Activity 6 – ATM Transaction Sequence Simulation (Case Study 13)

Assumptions

Assume the opening balance is Rs. 5,000. The transaction amount must be positive. Deposits have no stated limit; withdrawals are limited to Rs. 2,000 per transaction and cannot exceed the available balance.

Activity 7 – Student Performance Trend Analysis (Case Study 17)

Problem statement: Compare marks in two examinations and report improvement, decline, or unchanged marks.

Algorithm

1. Start.

2. Input first-exam marks for Python, Mathematics, and Physics.

3. Input second-exam marks for the same subjects.

4. For each subject, calculate difference = second-exam marks – first-exam marks.

5. Report improved, declined, or unchanged. Stop.

Pseudocode

```
START
INPUT first-exam marks for each subject
INPUT second-exam marks for each subject
FOR each subject
    difference = second_mark - first_mark
    IF difference > 0
        PRINT "Improved"
    ELSE IF difference < 0
        PRINT "Declined"
    ELSE
        PRINT "Unchanged"
    END IF
END FOR
STOP
```

Python Program

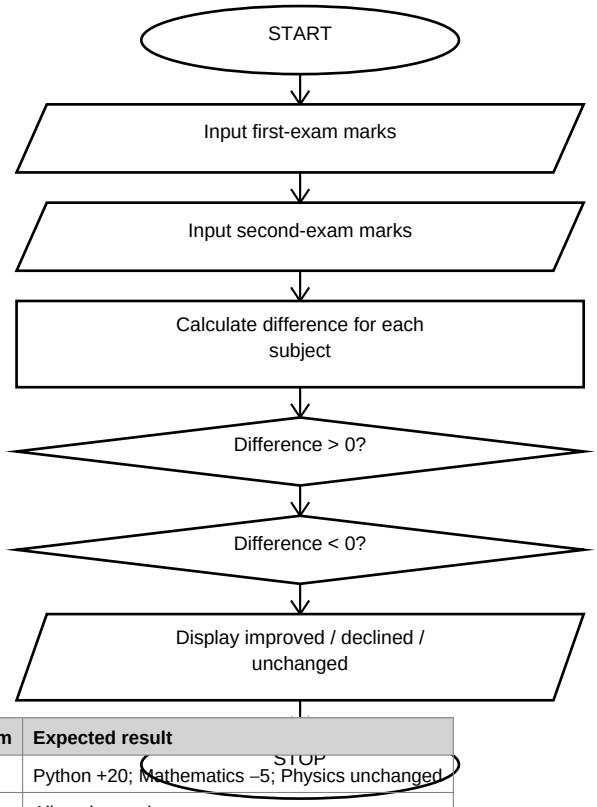
```
subjects = ["Python", "Mathematics", "Physics"]
exam1 = []
exam2 = []

for subject in subjects:
    exam1.append(int(input("Enter " + subject + " first exam marks: ")))
for subject in subjects:
    exam2.append(int(input("Enter " + subject + " second exam marks: ")))

for i in range(len(subjects)):
    difference = exam2[i] - exam1[i]
    if difference > 0:
        print(subjects[i], "improved by", difference, "marks")
    elif difference < 0:
        print(subjects[i], "declined by", abs(difference), "marks")
    else:
        print(subjects[i], "marks are unchanged")
```

Flowchart (standard symbols)

Oval = Start/Stop; parallelogram = input/output; rectangle = processing; diamond = decision.



Test Cases

First exam (Py, Math, Phy)	Second exam	Expected result
60, 70, 80	80, 65, 80	Python +20; Mathematics –5; Physics unchanged
50, 50, 50	50, 50, 50	All unchanged
90, 40, 75	70, 60, 80	Python –20; Mathematics +20; Physics +5

Sample Output (Text Format)

```
Enter Python first exam marks: 60
Enter Mathematics first exam marks: 70
Enter Physics first exam marks: 80
Enter Python second exam marks: 80
Enter Mathematics second exam marks: 65
Enter Physics second exam marks: 80
Python improved by 20 marks
Mathematics declined by 5 marks
Physics marks are unchanged
```

Conclusion

Compare marks in two examinations and report improvement, decline, or unchanged marks. The program demonstrates the Python concepts required for this activity.

Activity 7 – Assumptions

Program

Activity 7 – Student Performance Trend Analysis (Case Study 17)

Assumptions

Assume marks are entered as whole numbers on the same scale for both examinations. Compare each subject's second-exam mark with its first-exam mark; do not apply any mark limits or weighting.

Activity 8 – Advanced Numeric Pattern Generator (Case Study 27)

Problem statement: Generate a symmetric numeric pattern using nested loops and validate the number of rows.

Algorithm

1. Start.
2. Input number of rows n.
3. If n is outside 1–9, display invalid input.
4. Otherwise, for each row print leading spaces, increasing numbers, and decreasing numbers.
5. Stop.

Pseudocode

```
START
INPUT n
IF n < 1 OR n > 9
    PRINT "Invalid input"
ELSE
    FOR i from 1 to n
        PRINT n-i spaces
        PRINT numbers from 1 to i
        PRINT numbers from i-1 down to 1
        MOVE to next line
    END FOR
END IF
STOP
```

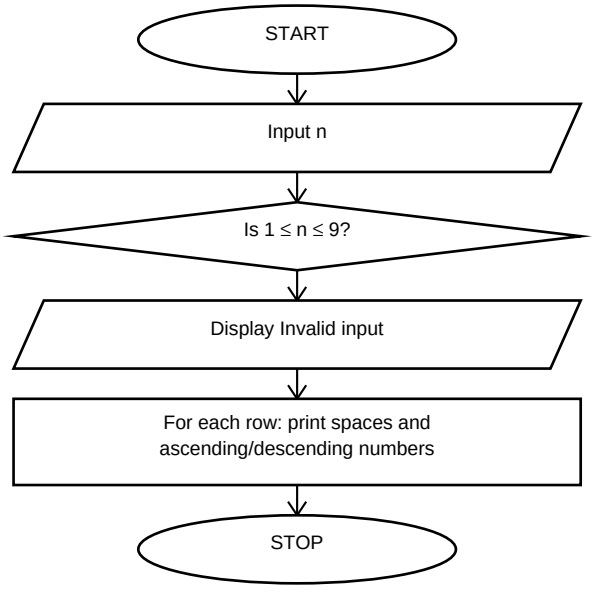
Python Program

```
n = int(input("Enter number of rows (1-9): "))

if n < 1 or n > 9:
    print("Invalid input")
else:
    for i in range(1, n + 1):
        print(" " * (n - i), end="")
        for j in range(1, i + 1):
            print(j, end="")
        for j in range(i - 1, 0, -1):
            print(j, end="")
        print()
```

Flowchart (standard symbols)

Oval = Start/Stop; parallelogram = input/output; rectangle = processing; diamond = decision.



Test Cases

Rows	Expected output
4	1 / 121 / 12321 / 1234321 (centered)
1	1
9	Valid 9-row pattern
0	Invalid input
10	Invalid input

Sample Output (Text Format)

```
Enter number of rows (1-9): 4
   1
  121
 12321
1234321
```

Conclusion

Generate a symmetric numeric pattern using nested loops and validate the number of rows. The program demonstrates the Python concepts required for this activity.

Activity 8 – Assumptions

Program

Activity 8 – Advanced Numeric Pattern Generator (Case Study 27)

Assumptions

Assume the input is an integer from 1 to 9. Any value outside this range is treated as invalid. Each row prints ascending numbers followed by descending numbers, centered with leading spaces.

Activity 9 – Student Marks Matrix Analysis (Case Study 42)

Problem statement: Store marks in a nested list, calculate totals and averages, and identify the top performer.

Algorithm

1. Start.
2. Store student names and marks in a nested list.
3. For each student, calculate total and average marks.
4. Find the index of the highest total.
5. Display each student's total and average and the top performer. Stop.

Pseudocode

```
START
STORE names and marks matrix
FOR each student
    total = sum of student's marks
    average = total / number of subjects
    STORE total
    PRINT total and average
END FOR
FIND index of maximum total
PRINT top performer and highest total
STOP
```

Python Program

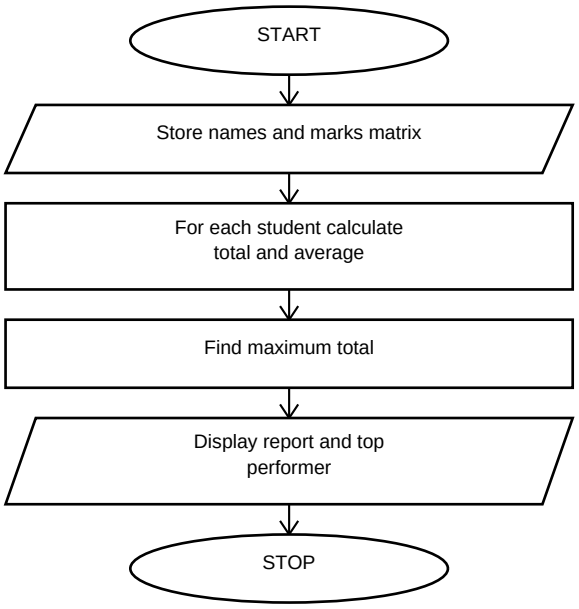
```
names = ["Ravi", "Priya", "Sita"]
marks = [
    [80, 75, 90],
    [95, 88, 92],
    [70, 85, 78]
]
totals = []

for i in range(len(marks)):
    total = sum(marks[i])
    average = total / len(marks[i])
    totals.append(total)
    print("Student:", names[i])
    print("Total marks:", total)
    print("Average:", round(average, 2))

top = totals.index(max(totals))
print("Top performer:", names[top])
print("Highest total:", totals[top])
```

Flowchart (standard symbols)

Oval = Start/Stop; parallelogram = input/output; rectangle = processing; diamond = decision.



Test Cases

Student marks	Expected total/average
80,75,90	245 / 81.67
95,88,92	275 / 91.67; top performer
70,85,78	233 / 77.67

Sample Output (Text Format)

```
Student: Ravi
Total marks: 245
Average: 81.67
Student: Priya
Total marks: 275
Average: 91.67
Student: Sita
Total marks: 233
Average: 77.67
Top performer: Priya
Highest total: 275
```

Conclusion

Store marks in a nested list, calculate totals and averages, and identify the top performer. The program demonstrates the Python concepts required for this activity.

Activity 9 – Assumptions

Program

Activity 9 – Student Marks Matrix Analysis (Case Study 42)

Assumptions

Assume the program uses the three students and three subject marks already listed in the code. Each subject has equal weight, and the top performer is the student with the highest total marks.

Activity 10 – Food Waste Monitoring System (Case Study 53)

Problem statement: Calculate leftovers and food-wastage percentage by comparing meals prepared with meals served over three days.

Algorithm

1. Start.

2. For Monday, Tuesday, and Wednesday, input meals prepared and meals served.

3. Validate prepared > 0 and 0 ≤ served ≤ prepared.

4. Calculate leftovers = prepared – served and wastage percentage = leftovers / prepared × 100.

5. If wastage > 10%, display excessive wastage; otherwise display acceptable wastage.

6. Repeat for all days and stop.

Pseudocode

```
START
FOR each day
    INPUT prepared, served
    IF prepared <= 0 OR served < 0 OR served > prepared
        PRINT "Invalid data"
    ELSE
        leftover = prepared - served
        percentage = leftover / prepared * 100
        PRINT leftover, percentage
        IF percentage > 10
            PRINT "Excessive wastage"
        ELSE
            PRINT "Acceptable wastage"
        END IF
    END IF
END FOR
STOP
```

Python Program

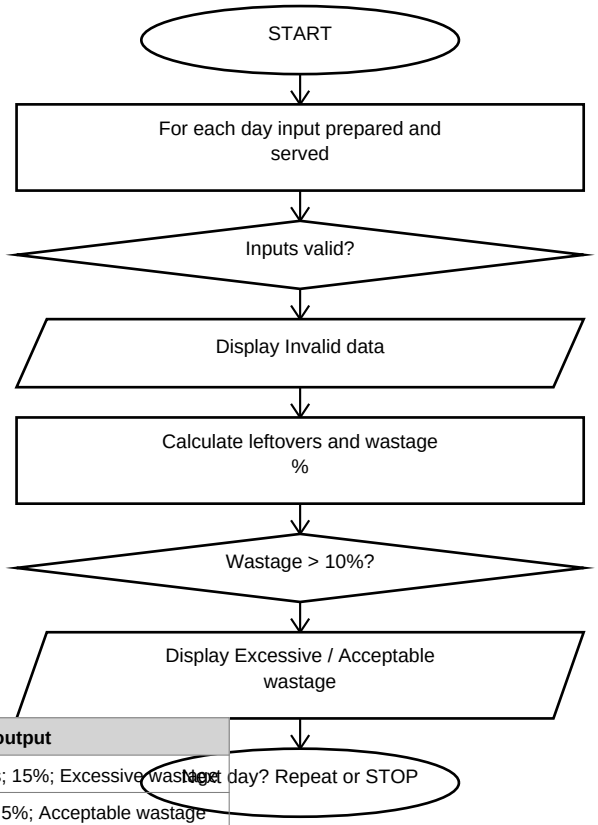
```
days = ["Monday", "Tuesday", "Wednesday"]

for day in days:
    prepared = int(input("Enter meals prepared on " + day + ": "))
    served = int(input("Enter meals served on " + day + ": "))

    if prepared <= 0 or served < 0 or served > prepared:
        print("Invalid data")
    else:
        leftover = prepared - served
        percentage = leftover / prepared * 100
        print("Day:", day)
        print("Leftover meals:", leftover)
        print("Wastage percentage:", round(percentage, 2), "%")
        if percentage > 10:
            print("Excessive food wastage")
        else:
            print("Acceptable food wastage")
```

Flowchart (standard symbols)

Oval = Start/Stop; parallelogram = input/output; rectangle = processing; diamond = decision.



Test Cases

Day data (prepared, served)	Expected output
100, 85	15 leftovers; 15%; Excessive wastage
100, 95	5 leftovers; 5%; Acceptable wastage
100, 100	0 leftovers; 0%; Acceptable wastage
0, 0	Invalid data
100, 120	Invalid data

Sample Output (Text Format)

```
Enter meals prepared on Monday: 100
Enter meals served on Monday: 85
Day: Monday
Leftover meals: 15
Wastage percentage: 15.0 %
Excessive food wastage
Enter meals prepared on Tuesday: 100
Enter meals served on Tuesday: 95
Day: Tuesday
Leftover meals: 5
Wastage percentage: 5.0 %
Acceptable food wastage
Enter meals prepared on Wednesday: 100
Enter meals served on Wednesday: 100
Day: Wednesday
Leftover meals: 0
Wastage percentage: 0.0 %
Acceptable food wastage
```

Conclusion

Calculate leftovers and food-wastage percentage by comparing meals prepared with meals served over three days. The program demonstrates the Python concepts required for this activity.

Activity 10 – Assumptions

Program

Activity 10 – Food Waste Monitoring System (Case Study 53)

Assumptions

Assume prepared and served meal counts are whole numbers for Monday, Tuesday, and Wednesday. Prepared meals must be greater than zero, and served meals must be between zero and the prepared count. Wastage above 10% is excessive.